

Semantic State After Native Reasoning

A Five-Turn Agentic Case Study of Jack XML, Context Projection, and Frozen-Answer Audit

Jonathan Langford

Independent Researcher | Lead Architect, The Jack Project

September 2026

Case-study result

Across a five-turn Agentic workload, the client-side session meter reached approximately 1.5M input tokens and 131k output tokens while displaying 147.4% against a 90k context budget. Jack performs its context projection after that client-side accounting step. The run therefore provides direct evidence of sustained cumulative work far beyond the nominal working context, while completed cognition was retired and selected semantic state was carried forward in Jack XML.

Evidence base: complete saved Pi/Jack run transcript, artifact files, CDP probe output, and final verification report.

Abstract

Long-running agentic work creates a simple systems problem: useful reasoning is expensive to keep alive, but throwing it away too early breaks continuity. A raw transcript preserves too much. A short summary often preserves too little, and it usually loses the distinction between what was observed, what was verified, what remains uncertain, and what was later found to be wrong.

This paper documents a five-turn Agentic run on Jack Kernel. The task was deliberately practical: inspect a broken analog-clock web page, build a corrected artifact, distill the important state, verify the result in a real browser runtime, and produce a final evidence-backed report. The run is useful because the third turn intentionally ended with the visible answer "DISTILLATION COMPLETE." Almost all of the useful work in that turn lived in the model's native reasoning and tool activity. Jack XML converted that transient work into a smaller, structured state before the raw cognition was retired.

The final turn produced the strongest result. Stage 1 completed the requested verification and correctly left the artifact unchanged, but it made a small factual error when describing the order and number of failed test-harness attempts. Stage 2, after the answer had already been frozen, reconstructed the tool history and caught the mistake. It did not rewrite the answer, and it did not turn a process-note error into an artifact failure. The correct 9/9 PASS verdict remained intact while the inaccurate narrative was recorded in the audit state.

The same run also reached a client-side ticker of "↑1.5M ↓131k 147.4%/90k." Because Pi displays that accounting before Jack performs its context projection and pruning, the number is not a 1.5M-token model context claim. It is evidence that cumulative work can continue far beyond the model's active working context when completed native cognition is retired and selected state is carried forward. The case study does not establish a universal accuracy gain. It does show, concretely, that native reasoning and durable semantic state can have different lifetimes and different authority.

1. Problem

Most agent frameworks treat conversation history as if it were memory. That works for short tasks. It becomes a liability during sustained work. A long technical session accumulates user instructions, source text, intermediate hypotheses, tool calls, failed commands, raw results, corrections, and final answers. Keeping all of it active indefinitely eventually collides with the model's context limit. Long before that hard limit, stale reasoning can interfere with new work.

The obvious alternative is summarization. But generic summaries flatten the parts of a technical investigation that matter most. A sentence supported by a file read is not the same thing as a model inference. A result reproduced in Chrome is not the same thing as a source-level guess. A premise that could invalidate a conclusion is not the same thing as an established defect. A mistake in an earlier answer should not silently disappear when that answer is summarized.

Jack Kernel treats this as a context-governance problem. The kernel sits between the application and the inference backend and decides what information is actually projected into each model call. In the Agentic program, rich native reasoning is allowed during substantive work. Once a Stage-1 answer is complete, that answer is frozen. A second stage sees the immediate Stage-1 cognition and produces a structured semantic record - Jack XML - with no authority to rewrite the answer.

The design question is not simply "How do we fit more history into context?" It is "What should still be allowed to influence the next inference?"

2. Agentic topology

The Agentic program used in this run separates substantive cognition from durable state extraction. Stage 1 does the work. Stage 2 records the parts of that work that should survive.

```

User turn
|
v
Stage 1 - substantive work
  native reasoning + tools + evidence
  |
  v
Freeze exact A1
  |
  v
Stage 2 - semantic distillation
  Preserve Thinking access to immediate Stage-1 cognition
  tools OFF | answer rewrite authority ZERO
  |
  v
Jack XML
  |
  v
Committed state: Jack XML + exact frozen A1

```

The separation matters because the two stages solve different problems. Stage 1 needs freedom: it can form hypotheses, use tools, encounter errors, change its mind, and work through details. Stage 2 needs discipline: it must decide what was actually grounded, what was verified, what still deserves challenge, and whether the frozen answer contains a concrete mistake.

2.1 The four semantic blocks

The current Jack XML used four blocks in this run. Grounding recorded observed or evidenced facts. Verification recorded what had actually been established and the scope of that verification. Challenge preserved live adversarial pressure - premises, hidden assumptions, and facts that would invalidate the conclusion. Audit recorded concrete mistakes already present in the frozen Stage-1 answer.

```

<Jack XML>
  <grounding> ... </grounding>
  <verification> ... </verification>
  <challenge> ... </challenge>
  <audit> ... </audit>
</Jack XML>

```

The tags are not valuable because they are XML. Their value comes from the fact that the state is typed. The model is not asked to produce one undifferentiated recap. It is asked to preserve different kinds of knowledge under different headings and different rules.

3. Native reasoning and durable state

Native reasoning is useful precisely because it is unconstrained. It contains local plans, discarded ideas, half-formed hypotheses, calculations, tool strategy, and self-correction. That is the right material to keep around while the model is still working. It is a poor long-term storage format once the task has moved on.

Jack XML is meant to be the opposite: lower-bandwidth, structured, and selective. The immediate Stage-1 reasoning can be treated as volatile transport. The distilled state can survive after that reasoning is retired. In practical terms, Preserve Thinking and Jack XML operate at different timescales. Preserve Thinking carries the full immediate cognitive trajectory into the next stage. Jack XML carries selected semantic state across the longer horizon.

Two timescales of state

Immediate adjacent-stage transport: Preserve Thinking - high bandwidth, volatile.

Long-horizon carry-forward: Jack XML - structured, semantic, durable.

4. The five-turn experiment

The task was intentionally mundane. That was useful. A browser clock is simple enough to inspect end to end, but it still provides source-level defects, geometry, arithmetic, runtime behavior, styling, tool failures, and a real verification boundary. The run therefore exercised long-horizon state without depending on an ambiguous benchmark score.

1. Turn 1 - Reconnaissance. Inspect the broken starter file and establish the defects from actual source evidence. No repair.
2. Turn 2 - Build. Create the corrected analog clock, preserve the requested formulas, and verify the written artifact.
3. Turn 3 - Distillation test. Re-read the evidence and the frozen artifact, re-establish the geometry failures and arithmetic, make no edits, and return exactly "DISTILLATION COMPLETE."
4. Turn 4 - Forensic verification. Use headless Chrome/CDP and deterministic checks to verify geometry, pinned-time behavior, layout, styling, and wraparound.
5. Turn 5 - Final verification. Because Turn 4 was 9/9 PASS, make no unnecessary repair, repeat the important deterministic checks, preserve evidence, and write a final report.

5. Turns 1 and 2: establishing the artifact

The starter file contained three important hand-geometry defects. The hand bars were horizontal rather than vertical. The hand block lacked left:50%, so it was not centered horizontally. Its transform origin was center top rather than the base of the hand. The file also lacked the requested title, internal label, date display, and luxury dark styling.

Turn 2 replaced that geometry with vertical hands positioned from the center of a 200 x 200 face. The hour, minute, and second hands used widths of 6, 4, and 2 pixels and heights of 30%, 40%, and 45%. Each used left:50%, a negative half-width offset, bottom:50%, and transform-origin:center bottom. The artifact retained the requested angle formulas and used a wall-clock-aligned setTimeout loop instead of a page-load-phase setInterval.

```
const h = now.getHours() % 12 || 12;
const secondDeg = (s / 60) * 360;
const minuteDeg = (m / 60) * 360 + (s / 60) * 6;
const hourDeg = (h / 12) * 360 + (m / 60) * 30;
```

At the end of Turn 2, the file existed and the required source structure was present. The deeper runtime question - whether the hands actually pivoted around the visual center in a browser and behaved correctly at exact times - remained for later turns.

6. Turn 3: the cleanest distillation result

Turn 3 was the most revealing part of the design. The model was told to do real work and then say almost nothing. It re-read the broken starter and the finished artifact, checked the exact geometry, and worked through the expected clock positions. It confirmed the original wrong pivot, horizontal axis, and missing centering; it

confirmed the corrected geometry in the frozen artifact; and it checked the 12, 3, 6, and 9 o'clock mappings from the actual formulas.

Then Stage 1 ended with the exact visible answer:

DISTILLATION COMPLETE

That answer is intentionally useless as a memory of the work. If the native reasoning were simply deleted after the turn, almost everything learned in Turn 3 would disappear with it. Jack XML changed that. Stage 2 received the full immediate Stage-1 trajectory, including the file reads, grep results, calculations, and frozen A1. It produced a compact record containing the source facts, corrected geometry, formula state, verification limits, and the three principal adversarial concerns.

This is the clearest evidence in the run that Jack XML is doing more than decorating the answer. The visible answer carried almost no state. The XML did.

6.1 What survived

The durable state preserved the starter's center-top pivot, the horizontal 30/40/45% geometry, the absence of left:50% in the hand block, the corrected bottom-center geometry in the built file, and the exact angle formulas. It also preserved the fact that the arithmetic had been model-computed from file-verified formulas, while browser rendering had not yet been observed. The challenge block kept the unresolved assumptions alive instead of presenting them as facts.

The Turn-3 information bottleneck

Visible A1: "DISTILLATION COMPLETE."

Durable semantic state: source defects, corrected geometry, formula state, verification boundaries, challenge conditions, and audit status.

The raw reasoning could now be retired without reducing the turn to two words.

7. Turn 4: moving from source claims to runtime evidence

Turn 4 deliberately raised the standard of proof. Static CSS was no longer enough. The model used headless Chrome 152 through the Chrome DevTools Protocol, injected pinned local times, inspected DOM geometry and computed transforms, checked the title/date/label layout, examined computed styling, and tested the second-boundary wraparound.

One intermediate measurement appeared to put the hand pivot at (98,98) instead of (100,100). The discrepancy came from mixing the face's border-box coordinates with offsets tied to the 196-pixel inner box created by the 2-pixel border. The model followed that discrepancy through the layout math instead of declaring an immediate artifact failure. A Node-side check produced the expected result: hour, minute, and second hand pivots all resolved to (100,100). Rendered rectangles at zero and ninety degrees independently agreed with that result.

The pinned-time results matched the formulas. At 12:00:00 the hands rendered vertically upward. At 15:00:15 the expected hour/minute/second angles were 90/1.5/90 degrees. At 10:10:30 they were 305/63/180 degrees. The title and date were centered with clear gaps from the clock face, the "simple clock" label remained inside the upper-central face area, and the computed styles contained the requested black, gold, and diamond treatment.

The wraparound check also passed. The second hand moved from 354 degrees at 23:59:59 to 0 degrees at 00:00:00. The source declared no transition, so there was no animated backward sweep. The angle logic was absolute rather than cumulative, which also removed a full-spin drift mode.

8. Turn 5: final verification under a no-repair rule

Turn 5 began from a 9/9 PASS result. The instruction was therefore conservative: do not edit a working artifact simply because a new turn had started. Stage 1 checked the 7,498-byte artifact and MD5 hash, confirmed the Reports directory, re-ran the static checks, and repeated the browser verification at the pinned times and wraparound boundary.

The artifact remained byte-identical. The final runtime results again matched the formulas to within a few ten-thousandths of a degree. The pivot offset was 0.0000 in both axes for all three hands. The title remained 42 pixels above the face, the date 46 pixels below it, and the label remained inside the upper-central region. A raw CDP evidence file and an 8,261-byte final verification report were written to the Reports directory.

8.1 Final verification results

Pinned time	Expected h/m/s	Observed	Result
12:00:00	360 / 0 / 0	rotate(360deg) / 0deg / 0deg; identity orientation	PASS
15:00:15	90 / 1.5 / 90	90 / 1.4999977 / 90	PASS
10:10:30	305 / 63 / 180	304.99998 / 63.000038 / 180	PASS
23:59:59 -> 00:00:00	359.5 / 359.9 / 354 -> 0 / 0 / 0	matched; no animated wraparound	PASS

9. The harness failed three times - the artifact did not

The final verification was messy in a useful way. The first helper-script attempt could not be found by Windows Node because the write tool and the shell did not agree on /tmp. The next attempt used a relative path that produced the wrong file URL and an empty page. A third attempt used a Date shim based on Date.call, which failed to initialize the native Date internal slot; the page script then stopped before setting the hands or date. Only after the harness was corrected with an absolute path and a class-based Date subclass did the runtime probe produce the expected values.

None of those failures implicated the clock. Stage 1 eventually distinguished the instrumentation problem from the system under test and completed the verification correctly. That distinction is important in agentic work: a failed tool or a bad probe is evidence about the probe until the artifact itself is implicated.

10. The strongest result: Stage 2 caught a real Stage-1 error

Stage 1 still made one mistake. In its final visible answer, it summarized the harness problems as "my first two probe runs" and associated them with the broken Date shim and the relative-path URL. The actual execution record contained three failed attempts before the passing run: the missing /tmp helper, the bad relative file URL, and the broken Date shim.

The error was small. It did not affect the artifact, any runtime measurement, the PASS verdict, or the conclusion that no repair was needed. But it was a real factual error in the frozen answer.

Stage 2 reconstructed the tool sequence, compared it with the frozen A1, and caught the mismatch. Its audit recorded the miscount and misordering while explicitly stating that the discrepancy did not affect any artifact claim or PASS/FAIL result. This is the cleanest example in the run of post-answer epistemic correction without answer rewriting.

Actual failed-harness sequence

1. /tmp helper path -> MODULE_NOT_FOUND
2. relative path -> wrong file:// URL -> face null
3. function Date shim -> 0-degree hands / empty date
4. corrected harness -> PASS

The important point is not that Stage 2 found a dramatic bug. It is that it found a narrow, real error, kept its scope narrow, and preserved the correct primary conclusion.

11. Why the audit result matters

A common multi-stage pattern is to let a later model rewrite an earlier answer. That can improve presentation, but it also hides provenance. Once a later stage replaces the answer, it becomes difficult to distinguish what the original cognition concluded from what the review stage changed.

Jack's Agentic program keeps those roles separate. The Stage-1 answer is frozen. Stage 2 can describe a mistake in that answer, but it cannot silently generate a replacement and treat the replacement as if it had always been the answer. In this case the durable state could simultaneously hold two truths: the analog-clock artifact passed all nine checks, and the final Stage-1 narrative misstated the harness-failure chronology.

That is a useful property for long-running work. The system does not have to choose between "the answer was correct" and "the answer contained an error." It can preserve the scope of both statements.

12. The 1.5M-token session marker

At the end of the saved run, Pi displayed the following session line:

```
~\Desktop\Jack Kernel
↑1.5M ↓131k 147.4%/90k
(jack-kernel) jack-kernel
```

This line needs careful interpretation. Pi computes and displays its own conversation accounting before Jack applies the program-specific pruning and context projection used for the backend request. The 147.4% figure therefore does not mean that a 1.5M-token prompt, or even a 132k-token prompt, was sent into a 90k model context. It means the client-side session history had grown beyond the nominal context indicator while Jack continued to build a bounded model-facing working set downstream.

The useful result is cumulative, not physical. The session accumulated roughly 1.5 million input tokens and 131 thousand output tokens of visible client-side traffic while the active backend context remained bounded. During that same run the model was still doing substantive work: reading files, running grep and hash checks, writing and debugging a CDP harness, launching browser probes, verifying geometry and exact clock angles, saving raw evidence, and performing the second-stage semantic audit.

What the ticker demonstrates

Cumulative work can greatly exceed the active model context. It does not demonstrate a 1.5M-token context window. The architectural point is that completed cognition can be retired after semantic distillation, allowing the session to continue without keeping the entire historical reasoning trajectory model-live.

The result pairs naturally with Turn 3. The run did not continue simply by discarding old work. The meaningful state of an intentionally minimal Stage-1 answer survived in Jack XML after the raw cognition was no longer needed. The ticker therefore illustrates the scale side of the design, while Turn 3 illustrates the continuity side.

13. Three material effects of Jack XML in this run

The case study supports three concrete observations. They are related, but they are not the same claim.

13.1 Semantic survival after pruning

Turn 3 directly demonstrates semantic survival. The visible answer was only "DISTILLATION COMPLETE." The useful content of the turn lived in the file reads, calculations, and reasoning. Stage 2 converted that transient work into a durable record before the native cognition could be retired. Without some state-extraction mechanism, pruning the native trajectory would have reduced the turn to two words and the user prompt.

13.2 Epistemic structure survived, not just facts

The XML preserved where claims came from and how strong they were. File content established by read and grep remained different from model-computed arithmetic. Source-level confidence remained different from browser-runtime verification. Unverified assumptions remained in challenge rather than being silently promoted to fact. This matters because the status of a fact is part of its operational meaning.

13.3 Frozen-answer auditing found something Stage 1 missed

The final Stage-2 audit identified a real narrative error in Stage 1. It did so after the answer was frozen, with no authority to rewrite it. The audit then kept the correction proportional: the process chronology was wrong, but the artifact verdict was still right. This is stronger evidence than simple state preservation because the second stage added a useful correction rather than only compressing what Stage 1 already knew.

14. What the experiment does not prove

This is a case study, not a controlled benchmark. It does not prove that Jack XML always improves answer quality. It does not establish a numerical effect size, a statistical significance level, or a universal reduction in hallucination. It also does not prove that XML syntax itself is responsible for the effect; another structured representation could in principle carry similar semantics.

Turn 4 is especially confounded as a quality comparison because the user prompt already specified what needed to be verified. A model without Jack XML might also have performed that verification well. The stronger evidence lies elsewhere: Turn 3 shows that substantive state survived despite an intentionally content-poor visible answer, and Turn 5 shows that Stage 2 found a real error in frozen Stage-1 output.

A clean causal test would hold the model, prompts, tools, sampling, pruning, and artifact constant while removing only the Stage-2 semantic state. That ablation would directly test whether later turns lose accuracy, repeat old mistakes, perform more redundant rereading, or show worse epistemic calibration when the XML is absent.

15. Proposed ablation

The existing five-turn task is unusually well suited to an XML-on/XML-off experiment because Turn 3 deliberately creates a severe information bottleneck.

```
Condition A - Jack XML ON
After Turn 3: user request + distilled Jack XML + "DISTILLATION COMPLETE"

Condition B - Jack XML OFF
After Turn 3: user request + "DISTILLATION COMPLETE"
(raw Stage-1 cognition retired in both conditions)
```

The later turns can then be compared on exact source recall, unnecessary tool rereading, contradiction rate, repeated-error rate, verification focus, artifact mistakes, distinction between harness and artifact failures, and final audit accuracy. The important control is that both conditions must retire the raw native cognition. Otherwise the experiment would measure Preserve Thinking or transcript retention rather than semantic distillation.

16. Context pruning as cognitive control

The broader point is that pruning should not be treated as a token-saving trick. It is an authority decision about which historical state remains causally active. A long session can contain far more cumulative work than the backend can process at once. The system therefore needs a policy for the lifetime of cognition.

Jack's policy in this program is straightforward: preserve active cognition while it is useful, distill completed cognition after the answer is frozen, then retire the raw trajectory while carrying forward the semantic state. That keeps the high-bandwidth reasoning where it has immediate value and avoids paying for it forever.

```
active cognition      -> preserve
completed cognition   -> distill
raw completed trajectory -> retire
semantic state        -> carry forward
```

This supports a relationship that is easy to misstate. Cumulative cognition can be much larger than the active model context, but cumulative cognition is not context length. The 1.5M-token session marker belongs to the cumulative-work side of that equation, not to the physical-context side.

17. Implications for local inference

The distinction is particularly useful for local models. A local model may have enough reasoning ability to do serious work but a context window that is small compared with the lifetime of a real project. The usual choices are to keep everything until the window fills, summarize repeatedly and accept growing semantic loss, or reset and lose state. Structured semantic carry-forward offers another option.

The model can reason richly inside an active stage, use tools, and consume substantial temporary context. Once that work is complete, the system can preserve a smaller record of the facts, evidence status, unresolved assumptions, and known mistakes. Later work can then proceed without pretending that the physical context window has expanded.

18. Discussion

The analog-clock task was not chosen because clocks are difficult. It was useful because the evidence was concrete enough to separate architectural effects from vague impressions of model quality. The starter was actually broken. The corrected artifact was actually written. Chrome actually rendered it. The pinned-time transforms were actually measured. The harness actually failed. The artifact actually remained unchanged. Stage 1 actually misstated the harness chronology. Stage 2 actually caught that error.

That sequence exposes the practical role of a durable semantic layer. It is not a replacement for native reasoning and it is not a replacement for tools. Native reasoning explores. Tools establish evidence. The frozen answer records what the substantive stage committed. Jack XML preserves the semantic and epistemic state that should still matter after the raw reasoning is no longer worth carrying.

The most useful outcome is therefore not the 9/9 PASS by itself. The artifact could have passed without Jack. The important outcome is that the system retained the right state across the full sequence while the session grew far beyond the active context budget, and that the review stage found a real, narrow error without being allowed to rewrite history.

19. Conclusion

This five-turn run provides a concrete demonstration of the Agentic program's central idea: native reasoning and durable semantic state do not need the same lifetime. Stage 1 can use the model at full cognitive bandwidth. Once

its answer is frozen, Stage 2 can extract a smaller, structured record of what was grounded, what was verified, what still deserves challenge, and what was wrong in the answer itself.

Turn 3 showed why that matters. The visible answer was intentionally reduced to "DISTILLATION COMPLETE," yet the substantive state survived. Turn 5 showed a different benefit: the semantic audit caught a real factual mistake in the frozen answer and scoped it correctly without disturbing the valid artifact verdict. The final Pi ticker - approximately 1.5M input tokens, 131k output tokens, and 147.4% against a displayed 90k budget - showed the same architecture operating across a session whose cumulative work had grown far beyond the active model context.

The evidence does not justify saying that Jack XML makes models universally smarter, or that Jack creates a larger context window. It supports a more precise claim: Jack Kernel can retire completed native cognition while preserving selected semantic and epistemic state, allowing long-running work to continue without keeping the entire reasoning history live. In this run, that state preservation was not passive. The final audit found a mistake that Stage 1 had missed.

The practical result is simple: the model can forget the path without forgetting what the path established.

Appendix A. Key evidence from the saved run

A.1 Turn 3 frozen answer

```
DISTILLATION COMPLETE
```

The surrounding Stage-1 trajectory contained the source reads, grep results, geometry reasoning, and clock-angle calculations. Stage 2 distilled those semantics into the durable Jack XML state before the raw cognition was retired.

A.2 Final Stage-1 verdict

```
Final status: PASS - 9/9. No repair required; artifact unmodified.
```

The final verification preserved the artifact at 7,498 bytes with MD5 9aaf8a7d00a13aa0659736f4f11655dd, and archived both the final report and raw CDP evidence.

A.3 Stage-2 audit finding

```
Harness narrative miscount/misorder: the frozen Stage-1 answer described two failed probe runs, but the tool record showed three failed attempts before the passing run. The discrepancy did not affect any artifact claim or PASS/FAIL verdict.
```

A.4 End-of-run Pi ticker

```
~\Desktop\Jack Kernel
↑1.5M ↓131k 147.4%/90k
(jack-kernel) jack-kernel
```

This is a client-side session-accounting observation. The ticker is displayed before Jack performs its backend-facing context projection. It should therefore be reported as cumulative session traffic/work, not as a model context length.

Appendix B. Evidence note

This paper is based on the complete saved local run, including the Pi transcript, Jack stage boundaries, tool calls and results, the generated analog-clock artifact, the final CDP evidence JSON, and the final verification report dated 10 September 2026. Numerical values and quoted execution details in the case study are drawn from that record. No claim in the paper requires the reader to accept the clock artifact as correct merely because the model said so; the final verdict was backed by static file checks, hashes, browser runtime probes, and pinned-time transform measurements.